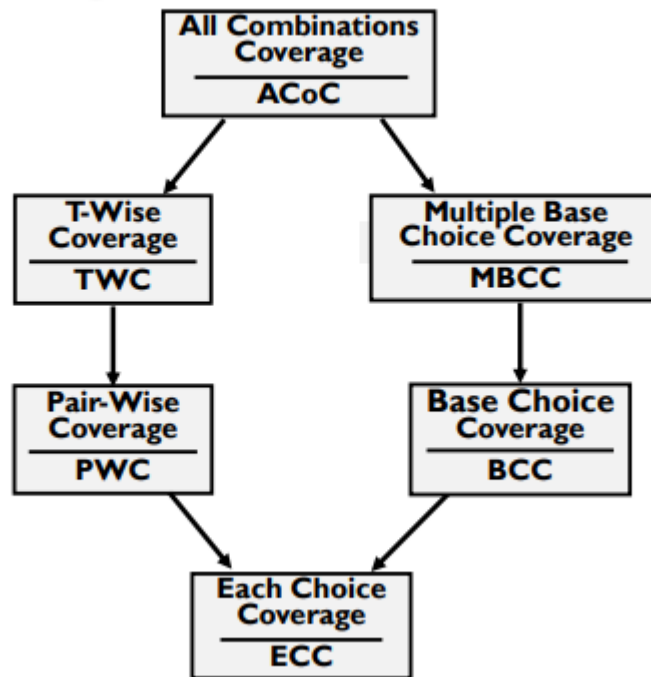


ISP Coverage Criteria Subsumption



1. Interface-Based Approach vs Functionality-Based Approach

Interface-Based Approach vs Functionality-Based Approach

Interface-Based Approach

Definition:

Interface-Based Approach is a technique in which the characteristics of the input domain are derived directly from the individual input parameters or syntax of the program interface.

Easy Meaning:

It creates input characteristics by looking at the individual input parameters.

Key Points:

- It is a simple modeling technique.
- Each input parameter is considered separately or in isolation.
- Relationships among input parameters are generally ignored.
- It mainly depends on the program's syntax or interface.
- It can be partially automated.
- It may produce an incomplete input domain model.

Example:

For a login system having **Username** and **Password**, the Username and Password may be considered as separate input characteristics.

Functionality-Based Approach

Definition:

Functionality-Based Approach is a technique in which the characteristics of the input domain are derived from the behavioral view of the program, its intended functionality, and its requirements.

Easy Meaning:

It creates input characteristics by considering how the program is expected to behave and what the requirements say.

Key Points:

- It considers the intended functionality of the system.
- It can consider relationships among input parameters.
- It uses domain and semantic knowledge.
- It requires more design effort from the tester.
- It is more difficult to develop than the Interface-Based Approach.
- It can produce more effective and meaningful test cases.

Example:

For a login system, this approach considers not only Username and Password separately, but also whether their combination is valid or invalid.

Comparison Table

Comparison Point	Interface-Based Approach	Functionality-Based Approach
Basis	Based directly on individual input parameters or program syntax.	Based on program behavior, intended functionality, and requirements.
Difficulty	Relatively simple and easy to apply.	More difficult and requires more design effort.

Input Parameters	Parameters are generally considered separately.	Parameters can be considered together.
Relationships	Usually ignores relationships among parameters.	Considers relationships among parameters.
Knowledge Used	Mainly uses interface and syntax information.	Uses domain, semantic, and requirement knowledge.
Automation	Can be partially automated.	Generally requires more manual design effort.
Test Cases	May produce less complete or less effective test cases.	Produces more effective and meaningful test cases.
Input Domain Model	May produce an incomplete model.	Can produce a more complete and realistic model.

In One Line

Interface-Based: Looks at what inputs the program has.

Functionality-Based: Looks at how those inputs should work together according to the requirements.

2. খুব সহজে পার্থক্য মনে রাখার উপায়

পরীক্ষার আগে শুধু এই ৪টা point মনে রাখলেও হবে:

বিষয়	Interface-Based	Functionality-Based
কী দেখে?	Input / Syntax	Function / Requirement
Parameter	আলাদাভাবে দেখে	একসাথে বিবেচনা করতে পারে
Relationship	Ignore করে	Consider করে
Result	Simple কিন্তু কম effective হতে পারে	Difficult কিন্তু বেশি effective

Memory Trick ☐

Interface = Input দেখো

Functionality = Function কীভাবে কাজ করে দেখো

অর্থাৎ,

Interface → What inputs are there?

Functionality → How should those inputs work?

3. Input Domain Modeling-এর ৫টি ধাপ

এটি পরীক্ষায় খুব সুন্দরভাবে Step-by-Step লিখতে পারো।

ধাপ ১ — Identify Testable Functions

প্রথমে নির্ধারণ করতে হবে কোন function, method, class, program বা system test করা হবে।

সহজ বাংলা:

কী test করব—সেটা প্রথমে ঠিক করতে হবে।

Example:

একটি Login System test করলে প্রথমে Login Function নির্বাচন করতে হবে।

ধাপ ২ — Find All the Parameters

Testable function-এর সঙ্গে সম্পর্কিত সব input parameters এবং relevant states/data identify করতে হবে।

এর মধ্যে থাকতে পারে:

- Method parameters
- Program state
- Global variables
- Files
- Database data
- System states

সহজ বাংলা:

Function-টি কাজ করার জন্য যেসব input বা data দরকার, সব খুঁজে বের করতে হবে।

ধাপ ৩ — Model the Input Domain

এটি Input Domain Modeling-এর সবচেয়ে creative এবং important design step।

এই ধাপে:

- Input domain-এর characteristics identify করা হয়।
- প্রতিটি characteristic-কে বিভিন্ন **blocks**-এ ভাগ করা হয়।
- প্রতিটি block একটি নির্দিষ্ট set of values represent করে।

সহজ বাংলা:

Input-এর কোন কোন বৈশিষ্ট্য আছে এবং প্রতিটি বৈশিষ্ট্যের কী কী possible value/group আছে— সেগুলো ভাগ করা হয়।

Example:

Age characteristic হলে:

- Negative age
- Valid age
- Very large age

এগুলো আলাদা blocks হতে পারে।

ধাপ ৪ — Apply a Test Criterion to Choose Combinations of Values

সব possible block-এর combination test করা প্রায় অসম্ভব, কারণ combination অনেক বেশি হয়ে যেতে পারে।

তাই একটি **test coverage criterion** ব্যবহার করে প্রয়োজনীয় combination নির্বাচন করা হয়।

সহজ বাংলা:

সব combination test করা সম্ভব নয়, তাই coverage criteria ব্যবহার করে গুরুত্বপূর্ণ combination বেছে নেওয়া হয়।

ধাপ ৫ — Refine Combinations of Blocks into Test Inputs

শেষ ধাপে selected block combinations থেকে **specific** এবং **appropriate values** নির্বাচন করে final test inputs তৈরি করা হয়।

সহজ বাংলা:

আগের ধাপে যে blocks নির্বাচন করেছি, সেখান থেকে আসল value নিয়ে final test input বানানো হয়।

Exam-Ready English Answer

Five Steps of Input Domain Modeling

Input Domain Modeling can be performed through the following five steps:

Step 1: Identify Testable Functions

First, identify the function, method, class, program, or system that needs to be tested.

Example:

For a login system, the Login function can be selected as the testable function.

Step 2: Find All the Parameters

Identify all parameters and relevant inputs that can affect the behavior of the selected function.

These may include:

- Method parameters
- Program state
- Global variables
- Files
- Database data
- System states

This step is generally straightforward.

Step 3: Model the Input Domain

This is the most creative design step of input domain modeling.

In this step, the input domain is structured according to its characteristics, and each characteristic is divided into different **blocks**. Each block represents a set of possible values.

Example:

For an Age characteristic, blocks may include:

- Negative age
- Valid age
- Extremely large age

Step 4: Apply a Test Criterion to Choose Combinations of Values

Testing all possible combinations of values is usually impractical because the number of combinations can become very large.

Therefore, an appropriate test criterion or coverage criterion is applied to select a suitable subset of combinations from the blocks.

Step 5: Refine Combinations of Blocks into Test Inputs

Finally, appropriate and specific values are selected from the chosen blocks and converted into actual test inputs.

Thus, the selected block combinations are transformed into executable test cases.

Summary

Step 1: Identify what to test.

Step 2: Find all inputs and parameters.

Step 3: Divide the input domain into characteristics and blocks.

Step 4: Select useful combinations using a test criterion.

Step 5: Convert the selected blocks into actual test inputs.

4. ৫টি ধাপের বাংলা Meaning একসাথে

Step	English	সহজ বাংলা
1	Identify Testable Functions	কী test করব তা নির্ধারণ করা
2	Find All the Parameters	সব input/parameter খুঁজে বের করা
3	Model the Input Domain	Input-কে characteristics ও blocks-এ ভাগ করা
4	Apply a Test Criterion	প্রয়োজনীয় block combinations নির্বাচন করা
5	Refine into Test Inputs	Blocks থেকে actual test input তৈরি করা

এক লাইনে পুরো process:

What to test → Find inputs → Make blocks → Choose combinations → Create test inputs

বাংলায়:

কী test করব → Input খুঁজি → Block বানাই → Combination বাছি → Final test input বানাই।

এভাবে লিখলে **সংজ্ঞা** + **explanation** + **comparison** + **৫টি ধাপ** + **example**—সবই cover হবে, তাই exam-এর জন্য বেশ strong answer হবে।

অবশ্যই। নিচের উত্তরটি **exam-ready**, সহজ English-এর সাথে **বাংলা meaning** দেওয়া হলো। পরীক্ষায় সরাসরি লিখতে পারবে।

Control Flow Graph (CFG), Simple Path and Prime Path

1. Basic Architecture of Control Flow Graph (CFG)

Definition

A **Control Flow Graph (CFG)** is a graphical representation of a program that shows the **flow of control or execution between different statements or blocks of statements**.

সহজ বাংলা:

Control Flow Graph বা CFG হলো একটি graph, যা program-এর এক statement থেকে অন্য statement-এ **কীভাবে execution flow করে** তা দেখায়।

Basic Components of CFG

A Control Flow Graph mainly consists of the following components:

1. Node

A **node** represents a single statement or a group/block of statements in a program.

বাংলা অর্থ:

Node একটি statement অথবা কয়েকটি statement-এর group/ব্লককে নির্দেশ করে।

Example:

$x = x + 1 \rightarrow$ একটি node হতে পারে।

2. Edge

An **edge** represents the flow of control from one node to another node.

বাংলা অর্থ:

Edge একটি node থেকে অন্য node-এ program-এর execution কীভাবে যায় তা নির্দেশ করে।

Example:

Node 1 $\longrightarrow$ Node 2

এখানে arrow-টি control flow নির্দেশ করে।

3. Entry Node

The **entry node** represents the point where the execution of the program or the selected function begins.

বাংলা অর্থ:

Entry node হলো যেখানে program বা function-এর execution শুরু হয়।

4. Decision Node

A **decision node** represents a condition where the program can follow **different execution paths**.

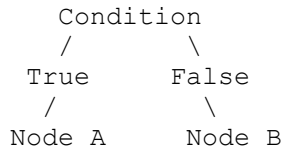
It commonly occurs in:

- if-else
- while
- for
- switch

বাংলা অর্থ:

Decision node হলো এমন একটি node যেখানে condition অনুযায়ী program-এর execution বিভিন্ন পথে যেতে পারে।

Example:



5. Exit Node

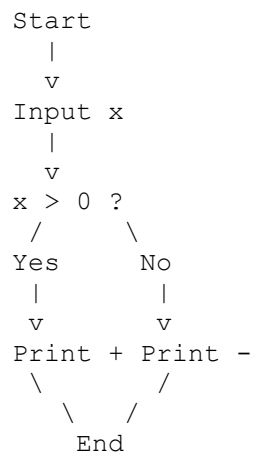
The **exit node** represents the point where the execution of the program or function terminates.

বাংলা অর্থ:

Exit node হলো যেখানে program বা function-এর execution শেষ হয়।

Simple CFG Example

ধরা যাক program:



এখানে:

- **Start** → Entry Node
 - **Input x** → Node
 - **x > 0?** → Decision Node
 - **Yes/No branches** → Edges
 - **End** → Exit Node
-

2. Simple Path

Definition

A **simple path** is a path in a Control Flow Graph in which **no node is repeated**, except that the first and last nodes may be the same when the path forms a cycle.

সহজ বাংলা:

CFG-এর এমন একটি path যেখানে কোনো node দ্বিতীয়বার আসে না, তাকে Simple Path বলে।

তবে যদি path-টি একটি cycle হয়, তাহলে প্রথম এবং শেষ node একই হতে পারে।

Example

$A \rightarrow B \rightarrow C \rightarrow D$

এখানে কোনো node repeat হয়নি।
তাই এটি একটি **Simple Path**।

আর:

$A \rightarrow B \rightarrow C \rightarrow B$

এখানে B repeat হয়েছে এবং first node A ও last node B একই নয়।
তাই এটি Simple Path নয়।

Cycle-এর ক্ষেত্রে:

$A \rightarrow B \rightarrow C \rightarrow A$

এখানে প্রথম ও শেষ node A একই। যেহেতু এটি একটি cycle, তাই এটি একটি **Simple Path** হতে পারে।

3. Prime Path

Definition

A **Prime Path** is a **simple path** that is **not a proper subpath of any other simple path**.

অর্থাৎ, এমন একটি **maximal simple path** যেটিকে আর কোনো বড় simple path-এর অংশ হিসেবে extend করা যায় না, তাকে Prime Path বলে।

সহজ বাংলা:

Prime Path হলো এমন একটি Simple Path, যেটি অন্য কোনো বড় Simple Path-এর proper অংশ নয়।

Example

ধরা যাক:

$A \rightarrow B \rightarrow C \rightarrow D$

এখানে:

$A \rightarrow B \rightarrow C$

একটি Simple Path। কিন্তু এটিকে D পর্যন্ত extend করা যায়:

$A \rightarrow B \rightarrow C \rightarrow D$

তাই $A \rightarrow B \rightarrow C$ একটি **Prime Path** নয়।

অন্যদিকে:

$A \rightarrow B \rightarrow C \rightarrow D$

যদি এটিকে আর কোনো বড় Simple Path-এ extend করা না যায়, তাহলে এটি **Prime Path**।

Simple Path vs Prime Path

বিষয়	Simple Path	Prime Path
Definition	No node is repeated.	A simple path that is not a proper subpath of another simple path.
Node Repetition	No node is repeated.	No node is repeated, because it is also a simple path.
Size	ছোট বা বড় যেকোনো simple path হতে পারে।	সাধারণত maximal simple path।
Extension	আরও বড় simple path-এর অংশ হতে পারে।	অন্য কোনো বড় simple path-এর proper অংশ হতে পারে না।
Relationship	Every prime path is a simple path.	Every prime path is simple, but every simple path is not prime.

সবচেয়ে গুরুত্বপূর্ণ লাইন ★

Every Prime Path is a Simple Path, but every Simple Path is not a Prime Path.

বাংলা:

প্রত্যেক Prime Path হলো Simple Path, কিন্তু প্রত্যেক Simple Path Prime Path নয়।

1. Coverage Criterion (কভারেজ ক্রাইটেরিয়ন)

- **English:** A rule or set of rules that specify what elements of a software artifact must be exercised by a test suite.
- **বাংলা অর্থ:** এক বা একাধিক নিয়মের সমষ্টি যা নির্ধারণ করে সফটওয়্যারের কোন অংশগুলো টেস্ট স্যুট দ্বারা যাচাই করতে হবে।
- **Figure / চিত্র:**

Plaintext

[Source Code / Model] ---> (Apply Coverage Rules) ---> [Selected Elements to Test]

- **Example / উদাহরণ:**
 - *EN:* For the expression $(x > 5)$, coverage requires testing both *true* and *false* outcomes.
 - *BN:* $(x > 5)$ এক্সপ্রেশনের জন্য ক্রাইটেরিয়ন হলো *true* এবং *false* উভয় ফলাফলই টেস্ট করা।

2. Test Criterion (টেস্ট ক্রাইটেরিয়ন)

- **English:** A property or standard that a test suite must meet to evaluate the thoroughness or completeness of testing.
- **বাংলা অর্থ:** একটি শর্ত বা আদর্শ যা টেস্ট স্যুটকে পূরণ করতে হয় টেস্টিংয়ের পুঙ্খানুপুঙ্খতা পরিমাপ করার জন্য।
- **Figure / চিত্র:**

Plaintext

[Test Suite Execution] ---> (Measure Against Standard) ---> [Thoroughness Evaluation]

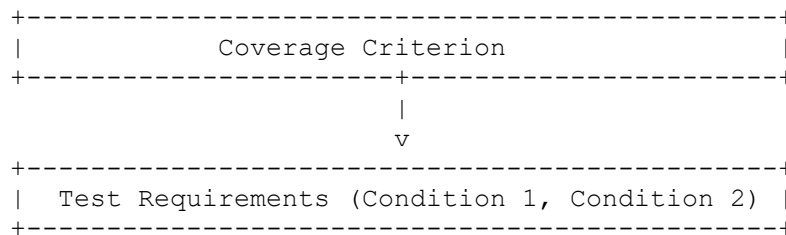
- **Example / উদাহরণ:**
 - *EN:* Statement Coverage requires every executable statement in the code to run at least once.

- *BN*: স্টেটমেন্ট কভারেজের নিয়ম হলো কোডের প্রতিটি এক্সিকিউটেবল স্টেটমেন্ট অন্তত একবার রান হতে হবে।

3. Test Requirements (টেস্ট রিকোয়ারমেন্টস)

- **English:** Specific, actionable conditions or structural components derived from a coverage criterion that must be satisfied by a test case.
- **বাংলা অর্থ:** কভারেজ ক্রাইটেরিয়ন থেকে প্রাপ্ত সুনির্দিষ্ট শর্ত বা উপাদান যা টেস্ট কেস দ্বারা অবশ্যই পূরণ করতে হবে।
- **Figure / চিত্র:**

Plaintext

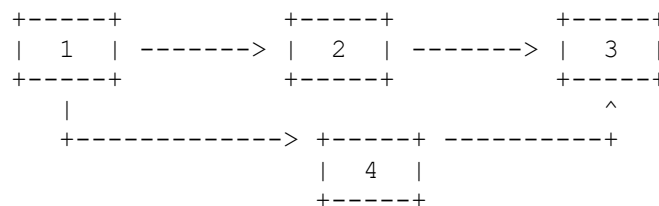


- **Example / উদাহরণ:**
 - *EN*: For Node Coverage on nodes {1, 2}, the requirements are to visit node 1 and visit node 2.
 - *BN*: {1, 2} নোডের ক্ষেত্রে নোড কভারেজের রিকোয়ারমেন্ট হলো নোড ১ এবং নোড ২ ভিজিট করা।

4. Node Coverage (নোড কভারেজ)

- **English:** A structural coverage criterion requiring the test suite to visit every node in a Control Flow Graph (CFG) at least once.
- **বাংলা অর্থ:** স্ট্রাকচারাল টেস্টিংয়ের এমন একটি ক্রাইটেরিয়ন যেখানে টেস্ট স্যুটকে কন্ট্রোল ফ্লো গ্রাফের (CFG) প্রতিটি নোড অন্তত একবার অতিক্রম করতে হয়।
- **Figure / চিত্র:**

Plaintext



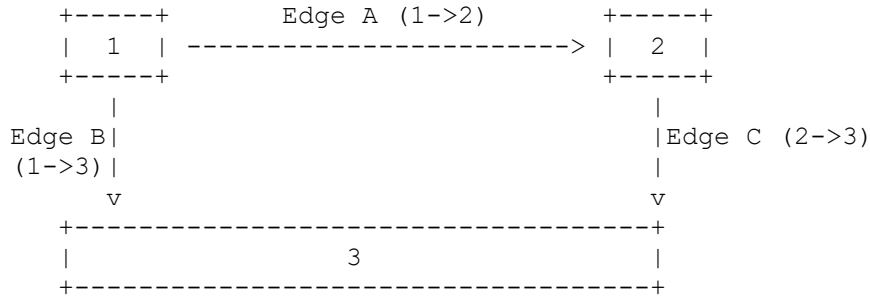
- **Example / উদাহরণ:**

- *EN*: Test paths must visit nodes {1, 2, 3, 4} using paths like [1, 2, 3] and [1, 4, 3].
- *BN*: টেস্ট পাথগুলোতে {1, 2, 3, 4} নোডগুলো অন্তর্ভুক্ত থাকতে হবে, যেমন— [1, 2, 3] এবং [1, 4, 3]।

5. Edge Coverage (এজ কভারেজ)

- **English:** A structural coverage criterion requiring the test suite to traverse every directed edge (transition between nodes) in a CFG at least once.
- **বাংলা অর্থ:** এমন একটি ক্রাইটেরিয়ন যেখানে টেস্ট স্যুটকে কন্ট্রোল ফ্লো গ্রাফের প্রতিটি ডিরেক্টেড এজ বা তীরচিহ্ন (নোডের মধ্যকার সংযোগ) অন্তত একবার অতিক্রম করতে হয়।
- **Figure / চিত্র:**

Plaintext

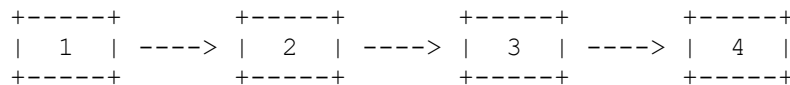


- **Example / উদাহরণ:**
 - *EN*: All edges { (1,2), (1,3), (2,3) } must be executed by the test suite.
 - *BN*: টেস্ট স্যুট দ্বারা { (1,2), (1,3), (2,3) } সবগুলো এজ বা সংযোগই রান করাতে হবে।

6. Edge-Pair Coverage (এজ-পেয়ার কভারেজ)

- **English:** A structural coverage criterion requiring the test suite to traverse every possible sequence of two adjacent edges (paths of length 2) in a CFG.
- **বাংলা অর্থ:** এমন একটি ক্রাইটেরিয়ন যেখানে টেস্ট স্যুটকে কন্ট্রোল ফ্লো গ্রাফের পাশাপাশি থাকা দুটি এজের সম্ভাব্য প্রতিটি জোড়া (দৈর্ঘ্য-২ এর সাবপাথ) অন্তত একবার অতিক্রম করতে হয়।
- **Figure / চিত্র:**

Plaintext



```

<--- Edge Pair 1 --->
      <--- Edge Pair 2 --->

```

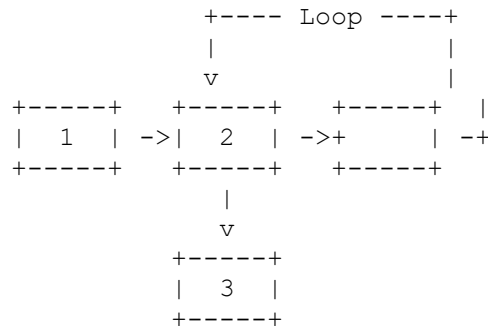
• **Example / উদাহরণ:**

- **EN:** Subpaths of length 2 like [1, 2, 3] and [2, 3, 4] must be covered by tests.
- **BN:** [1, 2, 3] এবং [2, 3, 4] এর মতো দৈর্ঘ্য-২ এর সাবপাথগুলো টেস্টে কভার করতে হবে।

7. Complete Path Coverage (কমপ্লিট পাথ কভারেজ)

- **English:** A structural coverage criterion requiring the test suite to execute every single possible execution path through the graph, including all loop iterations (zero, one, and multiple times).
- **বাংলা অর্থ:** এন্ট্রি নোড থেকে এক্সিট নোড পর্যন্ত যাওয়া সম্ভব এমন প্রতিটি সম্ভাব্য এক্সিকিউশন পাথ (লুপ শূন্য বার, এক বার বা বহুবার ঘোরার ক্ষেত্রসহ) টেস্ট করার ক্রাইটেরিয়াম।
- **Figure / চিত্র:**

Plaintext



• **Example / উদাহরণ:**

- **EN:** Requires test cases for loop iterations: 0 times ([1, 3]), 1 time ([1, 2, 3]), and 2 times ([1, 2, 2, 3]).
- **BN:** লুপের ক্ষেত্রে লুপ ০ বার ([1, 3]), ১ বার ([1, 2, 3]), এবং ২ বার ([1, 2, 2, 3]) ঘোরার জন্য আলাদা টেস্ট কেস প্রয়োজন।